

Architecture logicielle & systèmes distribués

[Accueil](#)[Introduction](#)[Jour 1](#)[Jour 2](#)[Jour 3](#)[Jour 4](#)[Jour 5](#)[Jour 6](#)[TP1](#)[TP2](#)[TP3](#)[Exercices](#)[Corrigés](#)[Glossaire](#)[Live Coding](#)[Dossier Architecture](#)[FAQ](#)[Support HTML modulaire](#)[Master 2](#)[6 jours](#)[3 TP progressifs](#)[Java / Spring Boot](#)

Dossier d'architecture

Objectifs pédagogiques

Apprendre à produire une documentation d'architecture utile, concise et exploitable.

Temps estimé

≈ 6h incluant démos et quiz

Prérequis

Jour 2 recommandé

Table des matières

- **Objectif**
- [Contenu](#)
- [Checklist](#)
- [ADR](#)
- [Mini dossier complet](#)

À quoi sert un dossier d'architecture ?

Le dossier d'architecture rend explicites les décisions structurantes, les contraintes, les risques et les compromis. Il sert autant à concevoir qu'à aligner équipes, parties prenantes et exploitation.

Mode enfant

C'est le cahier qui explique pourquoi on a construit la maison de cette manière et pas autrement.

Mode pro

Un bon dossier d'architecture réduit les décisions implicites, facilite l'onboarding, soutient la gouvernance et limite les débats sans mémoire.

Contenu attendu

- Contexte métier
- Besoins fonctionnels
- Besoins non fonctionnels
- Contraintes techniques / organisationnelles / réglementaires
- Choix d'architecture
- Alternatives rejetées
- Diagrammes à produire
- ADR
- Risques
- Trade-offs
- Checklist finale

Checklist d'architecture

- Le contexte métier est-il clair ?
- Les qualités prioritaires sont-elles explicites ?
- Le découpage est-il justifié ?
- Les flux synchrones et asynchrones sont-ils identifiés ?
- La sécurité est-elle abordée ?
- L'observabilité est-elle prévue ?
- Les risques et dettes acceptées sont-ils écrits ?

Exemple d'ADR

```
ADR-001 – Choix d'une API Gateway
Statut : accepté
Contexte : plusieurs services exposés à des clients front
Décision : introduire Spring Cloud Gateway
```

Conséquences positives : routage centralisé, sécurité, quotas, traçabilité

Conséquences négatives : nouveau point critique, compétence ops requise

Alternatives rejetées : accès direct client → services

Exemple complet de mini dossier d'architecture

Projet : Campus Library Flow

1. Contexte métier

Université multi-campus, 20 000 étudiants, consultation catalogue fréquente, création de prêts modérée, notifications email souhaitées.

2. Besoins fonctionnels

- rechercher un livre
- créer / retourner un prêt
- consulter l'historique d'un étudiant
- notifier un prêt créé

3. Besoins non fonctionnels

- disponibilité élevée pour la consultation catalogue
- traçabilité des prêts
- sécurité des opérations d'écriture
- temps de réponse < 500 ms sur la plupart des lectures

4. Contraintes

- équipe de 5 développeurs
- budget ops limité
- montée en charge attendue surtout sur la lecture

5. Choix

- monolithe propre au départ
- extraction ciblée du catalogue puis des prêts
- gateway pour exposition unifiée
- événement "loan-created" pour notification
- JWT pour sécuriser l'écriture

6. Alternatives rejetées

- microservices complets dès le début
- base partagée entre tous les services
- communication 100% synchrone

7. Risques

- complexité réseau croissante
- dette documentaire
- difficulté de test inter-services

8. Trade-offs

- plus de complexité technique
- meilleure évolutivité ciblée
- observabilité obligatoire

[← Live Coding](#)[FAQ →](#)